

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«Санкт-Петербургский государственный университет аэрокосмического приборостроения»

КАФЕДРА 33

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

Старший преподаватель	подпись, дата	К.А. Жиданов
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ
О ЛАБОРАТОРНОЙ РАБОТЕ VIBE-CODE:

по дисциплине: Технологии и методы программирования

РАБОТУ ВЫПОЛНИЛ

Студент гр. №	3331	подпись, дата	Ю.С. Брехунова
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2025

ЦЕЛЬ РАБОТЫ:

Изучение принципов создания системы авторизации на стороне сервера и реализация интеграции с Telegram-ботом, выводящим данные из базы данных (список задач). Работа направлена на освоение взаимодействия между клиентом, сервером и внешними API.

ХОД РАБОТЫ:

Данный проект представляет собой простое веб-приложение для управления задачами, разработанное на платформе Node.js, а также интегрированного с ним Telegram-бота. Такой функционал позволяет пользователям работать со списком задач как с компьютера, так и с мобильного устройства.

Задачи:

- Настроить сервер Node.js с поддержкой регистрации и входа пользователей.
- Реализовать работу с базой данных MySQL для хранения информации о пользователях и задачах.
- Создать Telegram-бота, который получает список задач из той же базы данных и отправляет его пользователю.

Структура проекта:

Имя	Дата изменения	Тип	Размер
 .gitignore	27.05.2025 23:18	Текстовый докум...	1 КБ
 ai.txt	27.05.2025 23:18	Текстовый докум...	0 КБ
 db.sql	08.06.2025 21:39	Исходный файл S...	1 КБ
 index.html	15.06.2025 0:39	Yandex Browser H...	5 КБ
 index.js	15.06.2025 0:55	Yandex Browser JS...	11 КБ
 login.html	08.06.2025 21:08	Yandex Browser H...	2 КБ
 package.json	27.05.2025 23:18	Исходный файл J...	1 КБ
 package-lock.json	27.05.2025 23:18	Исходный файл J...	5 КБ

Рис 1 – структура проекта

КОД ПРОЕКТА:

Код файла index.js:

```
const http = require('http');  
const fs = require('fs');
```

```
const path = require('path');
const mysql = require('mysql2/promise');
const crypto = require('crypto');
const TelegramBot = require('node-telegram-bot-api'); // Добавили
библиотеку для Telegram

const PORT = 3000;

const dbConfig = {
  host: 'localhost',
  user: 'root',
  password: 'yulchikzima',
  database: 'todolist',
  port: 3307,
};

function hashPassword(password) {
  return crypto.createHash('sha256').update(password).digest('hex');
}

function parseCookies(req) {
  const list = {};
  const cookieHeader = req.headers.cookie || '';
  cookieHeader.split(';').forEach(cookie => {
    const parts = cookie.split('=');
    list[parts.shift().trim()] =
      decodeURIComponent(parts.join('='));
  });
  return list;
}

async function retrieveListItems() {
```

```

    const connection = await mysql.createConnection(dbConfig);
    const [rows] = await connection.execute('SELECT id, text FROM
items');
    await connection.end();
    return rows;
}

```

```

async function getHtmlRows() {
    const todoItems = await retrieveListItems();
    return todoItems.map(item => `
        <tr>
            <td>${item.id}</td>
            <td id="text-${item.id}">${item.text.replace(/"/g,
'&quot;')}}</td>
            <td>
                <button onclick="deleteItem(${item.id})">x</button>
                <button onclick="editItem(${item.id})">✎</button>
            </td>
        </tr>
    `).join('');
}

```

```

async function handleRequest(req, res) {
    if (req.url === '/login' && req.method === 'GET') {
        try {
            const loginPage = await
fs.promises.readFile(path.join(__dirname, 'login.html'), 'utf8');
            res.writeHead(200, { 'Content-Type': 'text/html' });
            res.end(loginPage);
        } catch {
            res.writeHead(500, { 'Content-Type': 'text/plain' });
            res.end('Ошибка загрузки login.html');
        }
    }
}

```

```

    }
    return;
}

if (req.url === '/register' && req.method === 'POST') {
    let body = '';
    req.on('data', chunk => body += chunk.toString());
    req.on('end', async () => {
        try {
            const { username, password } = JSON.parse(body);
            if (!username || !password) {
                res.writeHead(400, { 'Content-Type': 'text/plain'
});
                res.end('Missing fields');
                return;
            }
            const connection = await
mysql.createConnection(dbConfig);
            try {
                await connection.execute('INSERT INTO users
(username, password) VALUES (?, ?)', [username,
hashPassword(password)]);
                res.writeHead(200, { 'Content-Type':
'application/json' });
                res.end(JSON.stringify({ status: 'registered' }));
            } catch {
                res.writeHead(400, { 'Content-Type': 'text/plain'
});
                res.end('User exists or error');
            } finally {
                await connection.end();
            }
        } catch {

```

```

        res.writeHead(400, { 'Content-Type': 'text/plain' });
        res.end('Invalid JSON');
    }
});
return;
}

if (req.url === '/login' && req.method === 'POST') {
    let body = '';
    req.on('data', chunk => body += chunk.toString());
    req.on('end', async () => {
        try {
            const { username, password } = JSON.parse(body);
            const connection = await
mysql.createConnection(dbConfig);
            const [rows] = await connection.execute(
                'SELECT * FROM users WHERE username = ? AND
password = ?',
                [username, hashPassword(password)]
            );
            await connection.end();

            if (rows.length === 1) {
                res.writeHead(200, {
                    'Content-Type': 'application/json',
                    'Set-Cookie':
`user=${encodeURIComponent(username)}; HttpOnly; Path=/`
                });
                res.end(JSON.stringify({ status: 'ok' }));
            } else {
                res.writeHead(401, { 'Content-Type':
'application/json' });
                res.end(JSON.stringify({ status: 'fail' }));
            }
        }
    });
}

```

```

        }
    } catch (error) {
        console.error('Login error:', error);
        res.writeHead(500, { 'Content-Type': 'text/plain' });
        res.end('Internal Server Error');
    }
});
return;
}

if (req.url === '/' && req.method === 'GET') {
    const cookies = parseCookies(req);
    if (!cookies.user) {
        res.writeHead(302, { Location: '/login' });
        res.end();
        return;
    }

    try {
        const html = await
fs.promises.readFile(path.join(__dirname, 'index.html'), 'utf8');
        const processedHtml = html.replace('{{rows}}', await
getHtmlRows());
        res.writeHead(200, { 'Content-Type': 'text/html' });
        res.end(processedHtml);
    } catch (err) {
        console.error(err);
        res.writeHead(500, { 'Content-Type': 'text/plain' });
        res.end('Error loading index.html');
    }
    return;
}

```

```

    if (req.url === '/add-item' && req.method === 'POST') {
      let body = '';
      req.on('data', chunk => body += chunk.toString());
      req.on('end', async () => {
        try {
          const { text } = JSON.parse(body);
          if (!text || text.trim() === '') {
            res.writeHead(400, { 'Content-Type': 'text/plain'
});
              res.end('Empty text');
              return;
            }
            const connection = await
mysql.createConnection(dbConfig);
            await connection.execute('INSERT INTO items (text)
VALUES (?)', [text.trim()]);
            await connection.end();
            res.writeHead(200, { 'Content-Type':
'application/json' });
            res.end(JSON.stringify({ status: 'success' }));
          } catch (err) {
            console.error('Error inserting item:', err);
            res.writeHead(500, { 'Content-Type': 'text/plain' });
            res.end('Server error');
          }
        });
      return;
    }
  }

```

```

    if (req.url === '/delete-item' && req.method === 'POST') {
      let body = '';
      req.on('data', chunk => body += chunk.toString());

```



```

req.on('end', async () => {
  try {
    const { id } = JSON.parse(body);
    if (!id) {
      res.writeHead(400, { 'Content-Type': 'text/plain'
});
      res.end('No ID provided');
      return;
    }
    const connection = await
mysql.createConnection(dbConfig);
    await connection.execute('DELETE FROM items WHERE id =
?', [id]);
    await connection.end();
    res.writeHead(200, { 'Content-Type':
'application/json' });
    res.end(JSON.stringify({ status: 'success' }));
  } catch (err) {
    console.error('Error deleting item:', err);
    res.writeHead(500, { 'Content-Type': 'text/plain' });
    res.end('Server error');
  }
});
return;
}

```

```

if (req.url === '/edit-item' && req.method === 'POST') {
  let body = '';
  req.on('data', chunk => body += chunk.toString());
  req.on('end', async () => {
    try {
      const { id, text } = JSON.parse(body);
      if (!id || !text || text.trim() === '') {

```

```

        res.writeHead(400, { 'Content-Type': 'text/plain'
    });

    res.end('Invalid ID or text');

    return;
  }

  const connection = await
mysql.createConnection(dbConfig);

    await connection.execute('UPDATE items SET text = ?
WHERE id = ?', [text.trim(), id]);

    await connection.end();

    res.writeHead(200, { 'Content-Type':
'application/json' });

    res.end(JSON.stringify({ status: 'success' }));
  } catch (err) {
    console.error('Error editing item:', err);
    res.writeHead(500, { 'Content-Type': 'text/plain' });
    res.end('Server error');
  }
});

return;
}

res.writeHead(404, { 'Content-Type': 'text/plain' });
res.end('Route not found');
}

// =====
// == Telegram Bot ==
// =====

const TELEGRAM_BOT_TOKEN =
'8093284051:AAFDNoQix2Vcrh4WhCo37quSaIMunSfpUas';

const bot = new TelegramBot(TELEGRAM_BOT_TOKEN, { polling: true });

```

```
bot.onText(/\start/, (msg) => {
    const chatId = msg.chat.id;
    bot.sendMessage(chatId, 'Привет! Напишите /tasks чтобы получить
    список задач.');
```

```
});

bot.onText(/\tasks/, async (msg) => {
    const chatId = msg.chat.id;
    try {
        const items = await retrieveListItems();
        if (items.length === 0) {
            bot.sendMessage(chatId, 'Задач пока нет.');
```

```
            return;
        }

        let response = '📋 Список задач:\n\n';
        items.forEach((item, i) => {
            response += `${i + 1}. ${item.text}\n`;
        });
```

```
        bot.sendMessage(chatId, response);
    } catch (err) {
        console.error('Ошибка при получении задач:', err);
        bot.sendMessage(chatId, 'Не удалось загрузить задачи.');
```

```
    }
});

// =====
// == Запуск сервера ==
// =====
```

```
const server = http.createServer(handleRequest);
server.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

Код файла index.html:

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width,
initial-scale=1.0">

  <title>To-Do List</title>

  <style>

    body {

      font-family: Arial, sans-serif;

    }

    #todoList {

      border-collapse: collapse;

      width: 70%;

      margin: 0 auto;

    }

    #todoList th, #todoList td {

      border: 1px solid #ddd;

      padding: 8px;

      text-align: left;

    }

    #todoList th {

      background-color: #f0f0f0;

    }

  </style>

</head>

<body>

  <div>

    <table>

      <thead>

        <tr>

          <th>Task</th>

          <th>Status</th>

        </tr>

      </thead>

      <tbody>

        <tr>

          <td>Task 1</td>

          <td>Completed</td>

        </tr>

        <tr>

          <td>Task 2</td>

          <td>In Progress</td>

        </tr>

        <tr>

          <td>Task 3</td>

          <td>Not Started</td>

        </tr>

      </tbody>

    </table>

  </div>

</body>

</html>
```

```

        #todoList th:first-child, #todoList th:last-child {
            width: 5%;
        }
        #todoList th:nth-child(2) {
            width: 90%;
        }
        .add-form {
            margin-top: 20px;
            width: 70%;
            margin: 20px auto;
        }
        .add-form input[type="text"] {
            padding: 8px;
            width: 70%;
        }
        .add-form button {
            padding: 8px;
            width: 20%;
        }
    </style>
</head>
<body>

<h2 style="text-align: center;">To-Do List</h2>

<table id="todoList">
    <thead>
        <tr>

```

```

        <th>Number</th>
        <th>Text</th>
        <th>Action</th>
    </tr>
</thead>
<tbody id="listBody">
    {{rows}}
</tbody>
</table>

<div class="add-form">
    <input type="text" id="newItem" placeholder="Enter new
item">
    <button onclick="addItem()">Add</button>
</div>

<script>
    async function addItem() {
        const newItemInput =
document.getElementById('newItem');
        const newItemText = newItemInput.value.trim();

        if (!newItemText) {
            alert('Введите текст');
            return;
        }

        try {
            const response = await fetch('/add-item', {

```

```

        method: 'POST',
        headers: { 'Content-Type':
'application/json' },
        body: JSON.stringify({ text: newItemText })
    });

    if (response.ok) {
        newItemInput.value = '';
        location.reload();
    } else {
        alert('Ошибка при добавлении элемента');
    }
} catch (e) {
    alert('Ошибка сети');
}
}

async function deleteItem(id) {
    if (!confirm('Удалить этот элемент?')) return;

    try {
        const response = await fetch('/delete-item', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ id })
        });

        if (response.ok) {
            location.reload(); // Обновить страницу после
удаления

```

```

        } else {
            alert('Ошибка при удалении элемента');
        }
    } catch {
        alert('Ошибка сети');
    }
}

async function deleteItem(id) {
    if (!confirm('Удалить этот элемент?')) return;

    try {
        const response = await fetch('/delete-item', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ id })
        });
        if (response.ok) location.reload();
        else alert('Ошибка при удалении элемента');
    } catch {
        alert('Ошибка сети');
    }
}

```

```

function editItem(id) {
    const td = document.getElementById(`text-${id}`);
    const oldText = td.textContent;

    td.innerHTML = `<input type="text" id="edit-input-${id}"
value="${oldText}" /> <button

```



```
onclick="saveEdit(${id})">Сохранить</button> <button  
onclick="cancelEdit(${id}, '${oldText}')">Отмена</button>`;  
}
```

```
function cancelEdit(id, oldText) {  
    const td = document.getElementById(`text-${id}`);  
    td.textContent = oldText;  
}
```

```
async function saveEdit(id) {  
    const input = document.getElementById(`edit-input-  
${id}`);  
    const newText = input.value.trim();  
    if (!newText) {  
        alert('Текст не может быть пустым');  
        return;  
    }  
  
    try {  
        const response = await fetch('/edit-item', {  
            method: 'POST',  
            headers: { 'Content-Type': 'application/json' },  
            body: JSON.stringify({ id, text: newText })  
        });  
        if (response.ok) {  
            location.reload();  
        } else {  
            alert('Ошибка при сохранении');  
        }  
    }  
}
```

```
    } catch {  
        alert('Ошибка сети');  
    }  
}
```

```
</script>
```

```
</body>
```

```
</html>
```

Код файла login.html:

```
<!DOCTYPE html>
```

```
<html lang="ru">
```

```
<head>
```

```
    <meta charset="UTF-8" />
```

```
    <title>Вход</title>
```

```
    <style>
```

```
        body { font-family: Arial, sans-serif; text-align:  
center; margin-top: 100px; }
```

```
        input { margin: 5px; padding: 8px; width: 200px; }
```

```
        button { padding: 8px 16px; }
```

```
    </style>
```

```
</head>
```

```
<body>
```

```
<h2>Вход</h2>
```

```
<form id="loginForm">
```

```
<input type="text" name="username" placeholder="Логин"
required /><br/>
```

```
<input type="password" name="password"
placeholder="Пароль" required /><br/>
```

```
<button type="submit">Войти</button>
</form>
```

```
<script>
```

```
const form = document.getElementById('loginForm');
form.addEventListener('submit', async (e) => {
  e.preventDefault();
  const data = {
    username: form.username.value.trim(),
    password: form.password.value.trim(),
  };
  try {
    const response = await fetch('/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(data),
    });
    if (response.ok) {
      window.location.href = '/'; // Перенаправляем на
главную страницу
    } else if (response.status === 401) {
      alert('Неверный логин или пароль');
    } else {
      alert('Ошибка сервера');
    }
  }
});
```

```
    } catch {  
        alert('Ошибка сети');  
    }  
});  
</script>  
  
</body>  
</html>
```

Код файла db.sql:

```
CREATE TABLE IF NOT EXISTS users (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    username VARCHAR(255) UNIQUE NOT NULL,  
    password VARCHAR(255) NOT NULL  
);
```

```
CREATE TABLE IF NOT EXISTS items (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    text VARCHAR(255) NOT NULL  
);
```

Краткое описание работы кодов:

Index.js:

Это серверный код на Node.js для To-Do List приложения с интеграцией Telegram-бота. Краткое описание:

Основные функции:

1. Веб-сервер (HTTP):

- Обработка запросов на порту 3000
- Маршруты для:

- Авторизации (/login)
- Регистрации (/register)
- CRUD операций с задачами (/add-item, /delete-item, /edit-item)
- Главной страницы (/)

2. Работа с БД (MySQL):

- Подключение к базе todolist
- Хеширование паролей (SHA-256)
- Запросы для управления задачами и пользователями

3. Telegram-бот:

- Команда /start - приветствие
- Команда /tasks - вывод списка задач из БД
- Использует токен бота

Index.html:

Это HTML-страница для веб-приложения To-Do List с JavaScript-функционалом.

Основные компоненты:

1. Вёрстка:

- Таблица для отображения списка задач (3 колонки: номер, текст, действия)
- Форма добавления новых задач (поле ввода + кнопка "Add")
- Заголовок "To-Do List" по центру страницы

2. Функционал (JavaScript):

- addItem() - добавляет новую задачу через POST-запрос на /add-item
- deleteItem(id) - удаляет задачу (подтверждение + запрос на /delete-item)
- editItem(id) - переводит задачу в режим редактирования
- saveEdit(id) - сохраняет изменения через /edit-item

- `cancelEdit(id, oldText)` - отменяет редактирование

3. Стили (CSS):

- Оформление таблицы (границы, отступы)
- Расположение формы добавления
- Подсветка заголовков таблицы

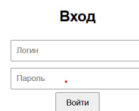
Login.html:

Этот код представляет собой **HTML-страницу с формой входа**, которая отправляет данные на сервер для аутентификации.

Db.sql:

Это SQL-код, который создает базу данных и таблицы для приложения списка задач (ToDo List).

ПРИМЕР РАБОТЫ ПРОГРАММЫ:



Вход

Логин

Пароль

Войти

Рис – 2. Авторизация

Вход



Войти

Рис 3 – Выполнение авторизации

localhost:3000 To-Do List Спросить

To-Do List

Number	Text	Action
3	Сходить в магазин	✕ ↘
4	Приготовить обед	✕ ↘
5	Прочитать книгу	✕ ↘
6	Сходить на пробежку	✕ ↘
7	Сделать лабораторную	✕ ↘

Add

Рис 4 – Список задач

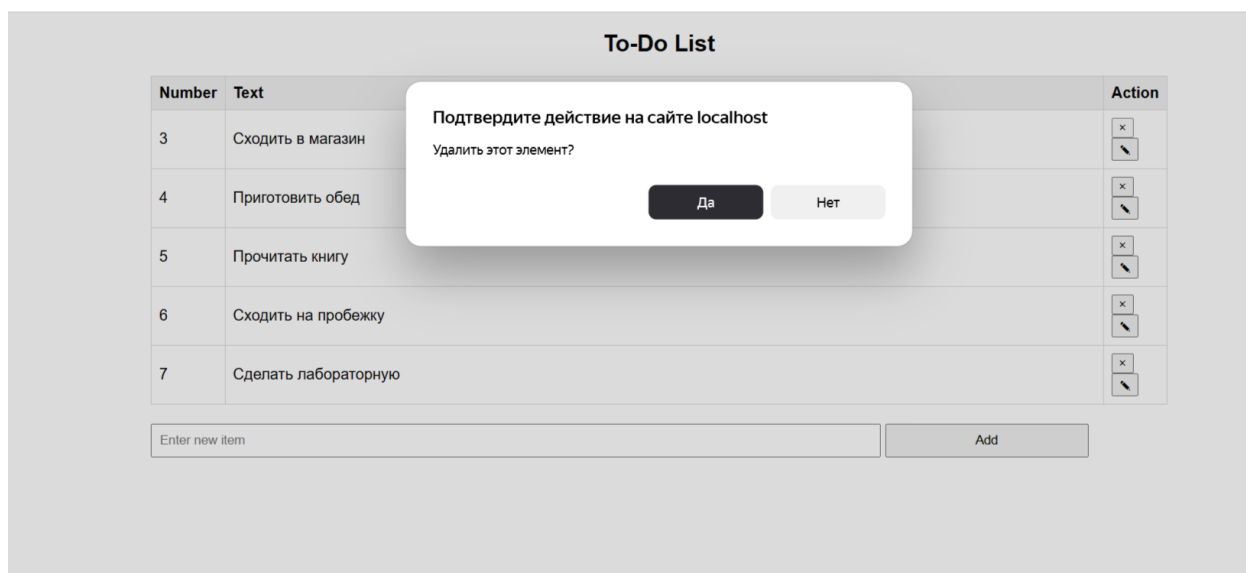


Рис 5 – Удаление элемента

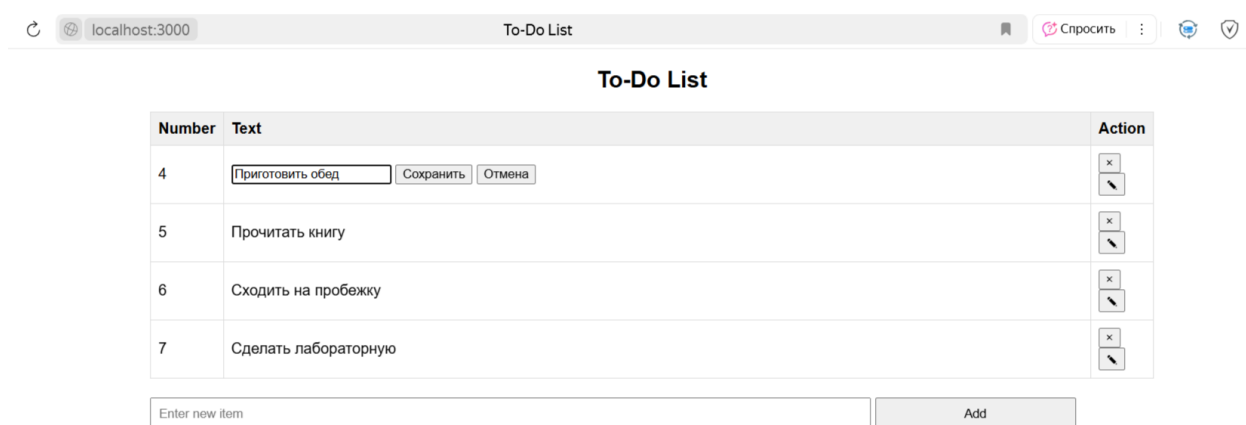


Рис 7 – Редактирование элемента

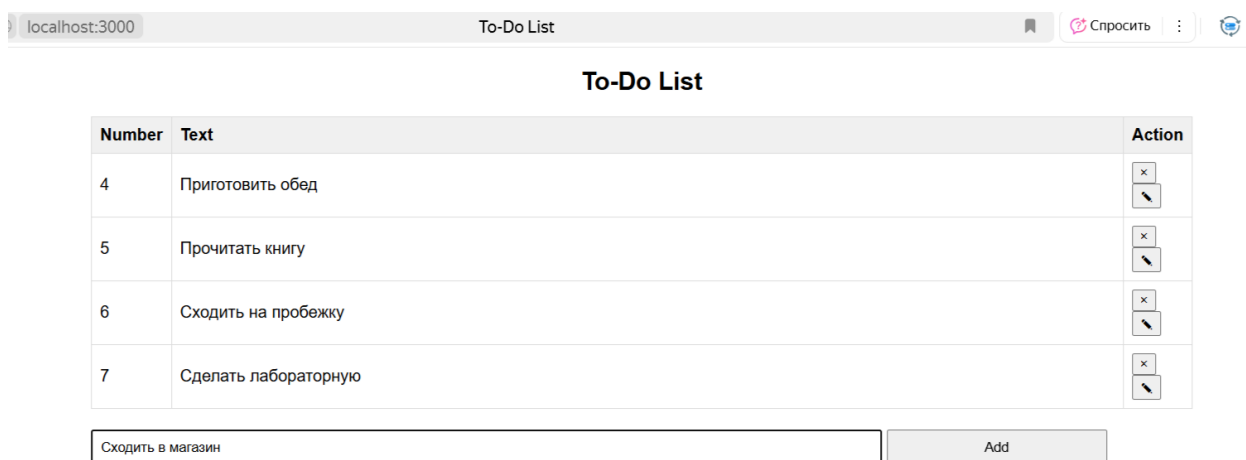


Рис 8 – Добавление элемента

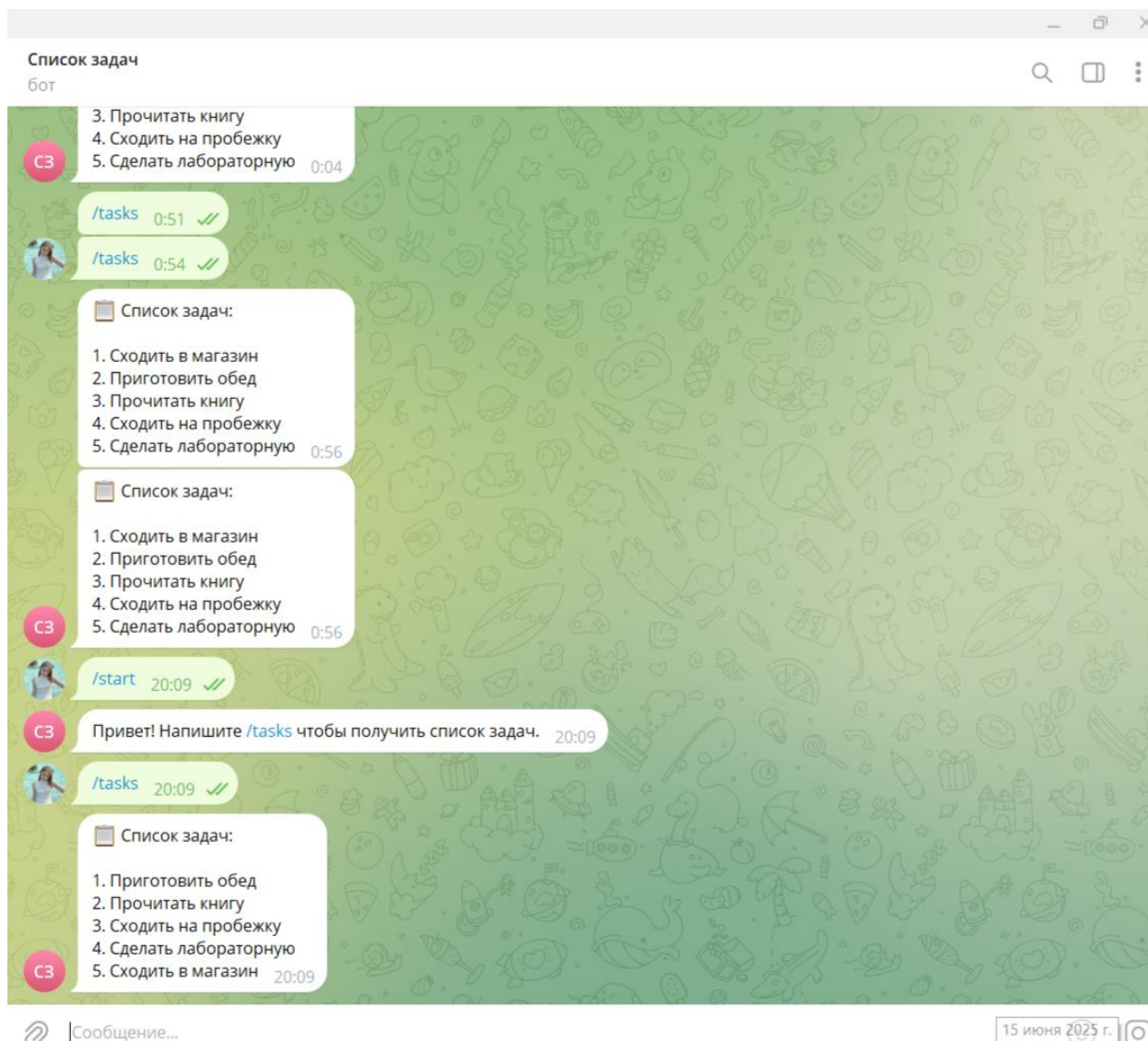


Рис 9 – Список задач от телеграмм бота

РАБОТА С НЕЙРОСЕТЬЮ:

Для данного проекта я использовала нейросеть ChatGPT.

В процессе выполнения данной работы нейросеть помогала исправлять ошибки, подробно объясняла пошаговый план действий для выполнения задач.

Примеры запросов для нейросети:

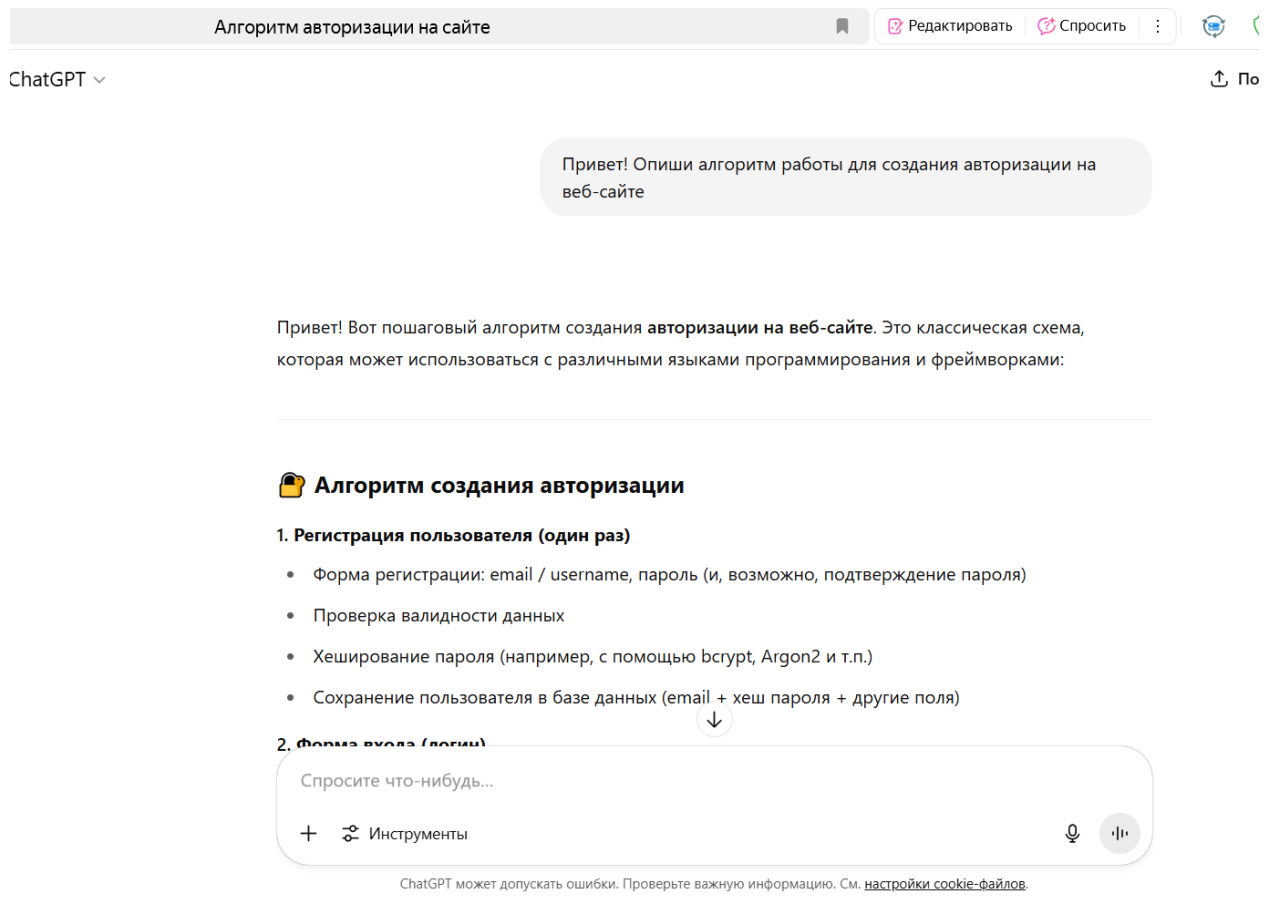


Рис 10 – Первый запрос

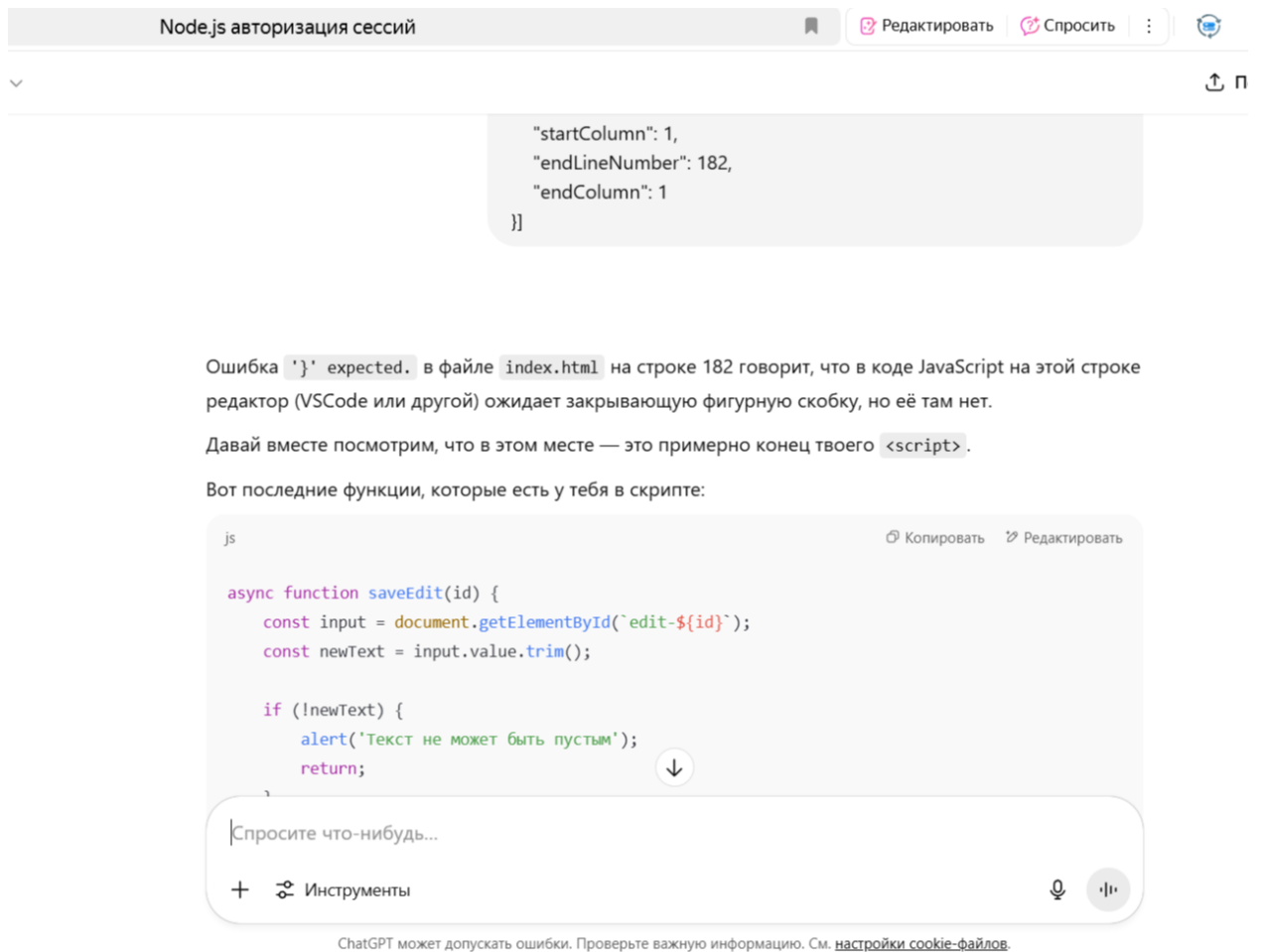


Рис 11 – Второй запрос

Исходя из примеров выше, можно сделать вывод о том, что с помощью нейросети (ChatGPT) удалось оперативно находить причины сбоев, корректировать как серверный, так и клиентский код, а также системно отлаживать процесс взаимодействия между фронтендом, сервером и базой данных. В результате приложение было успешно приведено к рабочему состоянию.

ВЫВОД:

В рамках разработки веб-приложения был реализован комплексный функционал, включающий авторизацию пользователей, работу с базой данных, интеграцию с Telegram-ботом, а также основные операции на добавление, редактирование и удаление задач. Работа велась с использованием Node.js, MySQL и JavaScript, что обеспечило прямой контроль над архитектурой приложения.

Проект продемонстрировал важность грамотного проектирования архитектуры, последовательной отладки и внимания к деталям. В процессе были решены реальные задачи, такие как:

- Передача cookie и проверка сессий;
- Интеграция стороннего API (Telegram);
- Работа с базой данных и обеспечение синхронности клиент-серверных операций.

Использование нейросети ChatGPT помогло быстро находить ошибки, улучшать структуру кода и принимать технически обоснованные решения. Этот опыт дал уверенность в работе с backend'ом, фронтендом и внешними сервисами, а также углубил понимание принципов безопасности и организации взаимодействия между компонентами веб-приложения.